

Détection automatique d'images

Classification chat / chien / animal sauvage
par un neurone artificiel

Rapport technique — Projet IA / Java / Traitement du Signal

Groupe : 8

Membres : Prénom Nom 1
Prénom Nom 2
Prénom Nom 3
Prénom Nom 4
Prénom Nom 5

Contents

1	Introduction et problématique	2
2	Niveau 1 — Maîtrise de l'objet neurone	2
2.1	L'algorithme d'apprentissage	2
2.2	Garde-fou contre la non-convergence	2
2.3	Validation sur les fonctions logiques ET et OU	2
3	Robustesse au bruit (analyse « signal »)	3
4	Niveau 2 — La chaîne de traitement	4
4.1	Architecture	4
4.2	Classification binaire (chat / pas chat)	4
5	Extensions	5
5.1	Classification multi-classes (one-vs-all)	5
5.2	Sur-apprentissage : effet du nombre d'itérations	5
5.3	Influence des hyperparamètres	6
5.4	Comparaison des représentations des données	6
6	Discussion : le plafond du modèle linéaire	7
7	Conclusion	7

1 Introduction et problématique

Ce projet vise à construire, en Java, une chaîne complète de traitement permettant de classer des images d'animaux à l'aide du neurone artificiel fourni. Le problème initial est binaire (*chat* / *pas chat*), puis étendu à trois classes (*chien*, *chat*, *animal sauvage*).

La démarche suit le protocole scientifique imposé : à partir des briques logicielles fournies (`iNeurone`, `Neurone`, `NeuroneHeaviside`, `Image`), nous posons une problématique, proposons des solutions, les mettons en œuvre et analysons les résultats de manière neutre et reproductible. Le jeu de données est celui du groupe 8, composé d'images couleur de résolution 64×64 pixels, réparties en un ensemble d'entraînement (*train*) et un ensemble de test (*test*) disjoints.

La problématique centrale est la suivante : **jusqu'où un unique neurone linéaire peut-il discriminer des images naturelles, et quels facteurs (fonction d'activation, prétraitement, représentation des données) gouvernent sa performance ?**

2 Niveau 1 — Maîtrise de l'objet neurone

2.1 L'algorithme d'apprentissage

Le neurone calcule une sortie $y = f(b + \sum_i w_i x_i)$, où x_i sont les entrées, w_i les poids synaptiques, b le biais et f la fonction d'activation. L'apprentissage fourni est une **règle delta** (descente de gradient simplifiée) qui répète, sur l'ensemble des exemples, la mise à jour suivante :

$$\delta = y_{\text{attendu}} - y_{\text{obtenu}}, \quad w_i \leftarrow w_i + \eta x_i \delta, \quad b \leftarrow b + \eta \delta \quad (1)$$

où η (**eta**) est le taux d'apprentissage. À chaque passe (*itération*), l'erreur quadratique moyenne (MSE) est recalculée ; la boucle s'arrête lorsque `mse` passe sous un seuil `MSElimite`. Le coefficient η contrôle la taille des pas de correction : trop grand, l'apprentissage oscille ou diverge ; trop petit, il converge lentement.

2.2 Garde-fou contre la non-convergence

La condition d'arrêt unique `while (mse > MSElimite)` ne se termine jamais si la MSE ne peut pas atteindre le seuil. Pour les fonctions d'activation continues ou les problèmes non linéairement séparables, ce cas est fréquent. Nous avons donc ajouté une **surcharge** de la méthode `apprentissage` acceptant un nombre maximal d'itérations `iterMax`, sans modifier la méthode d'origine :

```
public void apprentissage(final float[][] entrees, final float[] resultats,
                          final float MSElimite, final int iterMax) {
    double mse; int iter = 0;
    do {
        /* ... règle delta identique à la version d'origine ... */
        iter += 1;
    } while (mse > MSElimite && iter < iterMax); // double condition d'arret
}
```

2.3 Validation sur les fonctions logiques ET et OU

Les fonctions ET et OU sont linéairement séparables : un neurone unique peut donc les apprendre parfaitement. Nous avons validé les trois fonctions d'activation sur ces cas de référence. Le tableau 1 synthétise le comportement observé.

Table 1: Comportement des trois fonctions d'activation sur ET / OU.

Activation	Convergence	Itérations (ordre de grandeur)	MSE finale
Heaviside	Oui, exacte	$\sim 6\,500$ (ET et OU)	0 (exact)
ReLU	Oui, approchée	$\sim 15\,800$ (ET), $\sim 10\,600$ (OU)	plafonne au seuil
Sigmoïde	Oui, très lente	$\sim 1,6 \times 10^6$	plafonne au seuil

Heaviside. Sa sortie strictement binaire (0 ou 1) lui permet d'atteindre une MSE *exactement* nulle sur un problème séparable. Une étude statistique sur plusieurs exécutions montre toutefois une forte **instabilité géométrique** : les poids finaux varient radicalement d'un run à l'autre (de l'ordre de $0,19$ à 10^{-4} selon l'initialisation), car l'algorithme valide la première droite de séparation trouvée sans chercher d'optimum global. Dans de rares cas, l'initialisation aléatoire produit immédiatement une solution valide (0 itération).

ReLU — étude statistique sur 50 exécutions. Le tableau 2 présente les statistiques relevées ($\eta = 10^{-4}$, $\text{MSE}_{\text{limite}} = 0,1$).

Table 2: Statistiques ReLU sur 50 exécutions (fonctions ET et OU).

Fonction	Itér. moy.	Synapse 0	Synapse 1	Biais	Écart-type poids
ET	$\sim 15\,756$	0,1842	0,1767	-0,1156	$\pm 0,044$
OU	$\sim 10\,597$	0,2135	0,2254	+0,0495	$\pm 0,046$

ReLU converge bien plus vite que la sigmoïde mais reste instable : les écarts-types des poids ($\pm 0,04$) confirment qu'elle ne converge pas vers une solution unique. La fonction OU converge plus vite que ET (3 cas positifs sur 4 contre 1 sur 4, donc un signal d'erreur plus fréquent). Enfin, ReLU n'étant pas bornée, elle peut produire des sorties supérieures à 1, ce qui la rend peu adaptée à une classification binaire stricte en couche de sortie.

Sigmoïde. Sa sortie continue dans $(0,1)$ n'atteint 0 ou 1 que de façon asymptotique : la MSE plafonne au seuil et la convergence est extrêmement lente ($\sim 1,6$ million d'itérations), les poids étant poussés vers la saturation.

3 Robustesse au bruit (analyse « signal »)

Une fois le neurone ayant appris parfaitement la fonction ET, nous l'avons alimenté avec des entrées bruitées : à chaque essai, un bruit uniforme d'amplitude A est ajouté autour de 0 et 1. Le rapport signal/bruit est ici défini comme $\text{SNR} = 1/A$ (le signal valant 1, l'écart entre les niveaux logiques). La figure 1 compare Heaviside et ReLU.

Interprétation. Heaviside chute immédiatement à $\sim 75\%$ (3 cas sur 4 corrects) puis forme un plateau : sa sortie binaire bascule « tout ou rien » dès que le cas le plus proche de la frontière de décision (ici $\{1,1\}$, à faible marge) franchit le seuil. ReLU, dont la sortie est graduée, conserve une marge qui résiste aux faibles perturbations : sa précision décroît continûment. **Sur cette tâche, la réponse graduée (ReLU) est plus robuste au bruit que le seuillage binaire (Heaviside).** La robustesse n'est donc pas uniforme : elle dépend de la distance de chaque exemple à la frontière de décision.

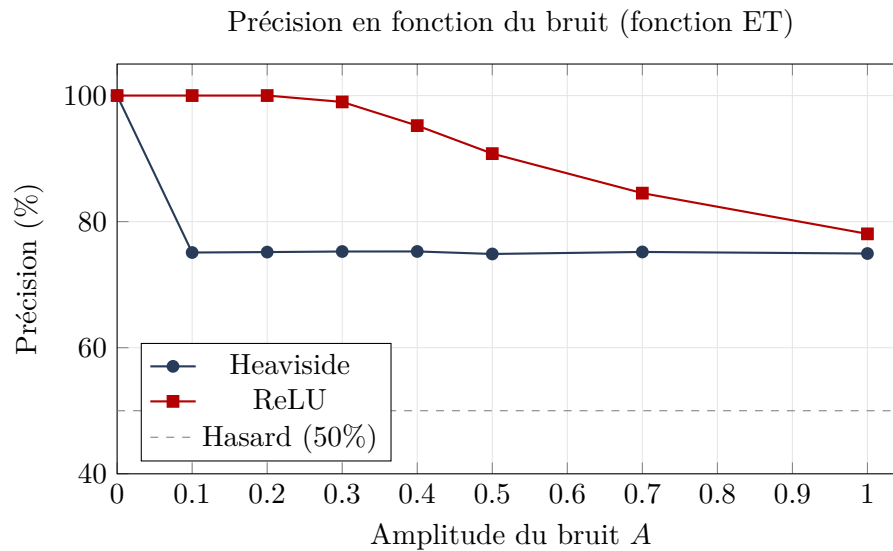


Figure 1: Robustesse au bruit : ReLU se dégrade progressivement, Heaviside chute brutalement puis stagne.

4 Niveau 2 — La chaîne de traitement

4.1 Architecture

La chaîne respecte l'ordre imposé :

1. lecture des fichiers d'entraînement (`Image.listeFichiers`) ;
2. labellisation à partir du nom du dossier (`cat`, `dog`, `wild`) ;
3. normalisation des pixels ($[0,255] \rightarrow [0,1]$) ;
4. mélange des données (`Collections.shuffle`) ;
5. entraînement du / des neurone(s) ;
6. lecture, labellisation et normalisation du jeu de test ;
7. application du modèle et calcul du taux de bonne classification.

L'image étant déjà aplatie en un vecteur 1D par la classe `Image`, le branchement sur le neurone est direct. Une vérification dimensionnelle écarte toute image dont la taille diffère de la référence.

4.2 Classification binaire (chat / pas chat)

Avec un neurone ReLU unique, les meilleurs résultats sur le jeu de test sont reportés au tableau 3. La référence du hasard pour un problème binaire est 50%.

Table 3: Classification binaire chat / pas chat (neurone ReLU).

Configuration	Itérations	MSE finale	Précision test
$\eta = 10^{-3}$, <code>MSElimite</code> = 0,10	2 095	0,100	64,14%
$\eta = 10^{-3}$, <code>MSElimite</code> = 0,08	12 959	0,080	65,76%
Sans mélange (<code>shuffle</code>)	466	0,050	48,17%
Sans normalisation	—	0,519 (bloquée)	divergence

Rôle du mélange. Le cas *sans mélange* est instructif : la MSE d'entraînement descend très bas (0,050 en 466 itérations) mais la précision de test s'effondre au niveau du hasard (48,17%). Les fichiers étant lus groupés par classe, le neurone voit successivement tous les exemples d'une catégorie puis de l'autre, et finit biaisé vers la dernière classe vue. **Le mélange est donc indispensable.**

Rôle de la normalisation. Sans normalisation, les entrées (de 0 à 255) font exploser le terme de correction $x_i \eta \delta$: la MSE se bloque (le neurone dégénère vers une sortie constante) et l'apprentissage diverge.

5 Extensions

5.1 Classification multi-classes (one-vs-all)

Un neurone n'ayant qu'une sortie binaire, la classification à trois classes est réalisée par la stratégie *un contre tous* : trois neurones sigmoïdes « experts » sont entraînés, chacun à reconnaître une classe (chien / chat / sauvage) contre les deux autres. Les mêmes images servent aux trois, avec des cibles différentes. À la prédiction, on retient l'expert au score le plus élevé. La sigmoïde est choisie car sa sortie bornée $[0,1]$ rend les trois scores directement comparables pour le vote — ce que ne permettraient ni Heaviside (égalités fréquentes) ni ReLU (sorties non bornées qui écraseraient le vote).

5.2 Sur-apprentissage : effet du nombre d'itérations

En représentation RGB ($\eta = 10^{-2}$), nous avons fait varier le nombre d'itérations. La figure 2 met en évidence un **sur-apprentissage** caractéristique.

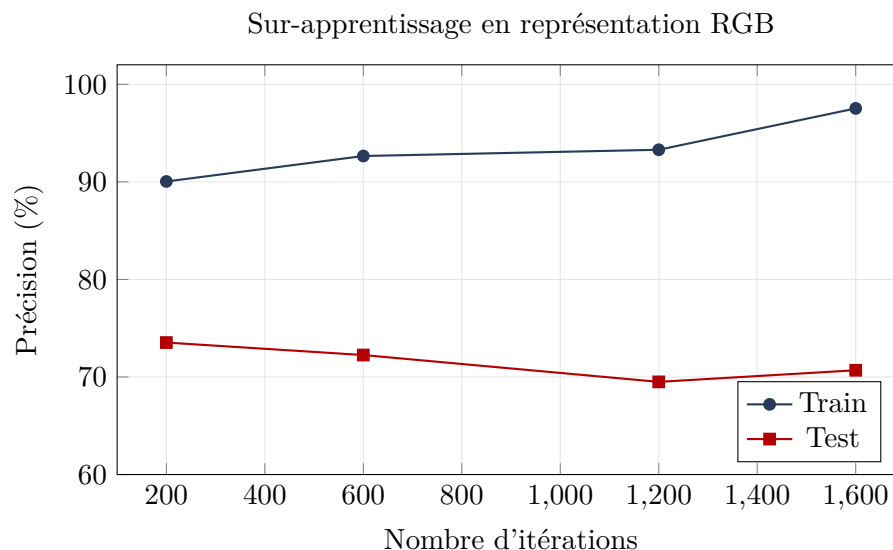


Figure 2: La précision d'entraînement augmente continûment tandis que la précision de test stagne, voire diminue : le modèle « mémorise » sans mieux généraliser.

L'écart croissant entre train ($90 \rightarrow 97,5\%$) et test ($\sim 73 \rightarrow 70\%$) est la signature du sur-apprentissage. La meilleure généralisation est obtenue avec *peu* d'itérations (200), ce qui plaide pour un arrêt précoce (*early stopping*).

5.3 Influence des hyperparamètres

Le balayage de η (de 10^{-2} à 10^{-4}), de `MSElimite` et du nombre d'itérations (jusqu'à 2000) laisse la précision de test confinée entre 70 et 72% en représentation TSL. Un petit η exige davantage d'itérations pour atteindre le même point, sans déplacer le plafond. **Aucun hyperparamètre ne franchit ce plafond**, ce qui suggère un facteur limitant structurel plutôt qu'un mauvais réglage.

5.4 Comparaison des représentations des données

Nous avons comparé six représentations en entrée du modèle multi-classes. Le tableau 4 et la figure 3 synthétisent les meilleurs résultats de test. La référence du hasard pour trois classes est $\approx 33\%$.

Table 4: Précision de test selon la représentation (modèle multi-classes, sigmoïde).

Représentation	Itérations	Train	Test
Niveaux de gris / RGB	200	90,04%	73,53%
TSL (HSL)	600	100,00%	71,84%
FFT 2D (spectre d'amplitude)	600	71,16%	63,09%
HOG (gradients orientés)	600	89,55%	86,88%

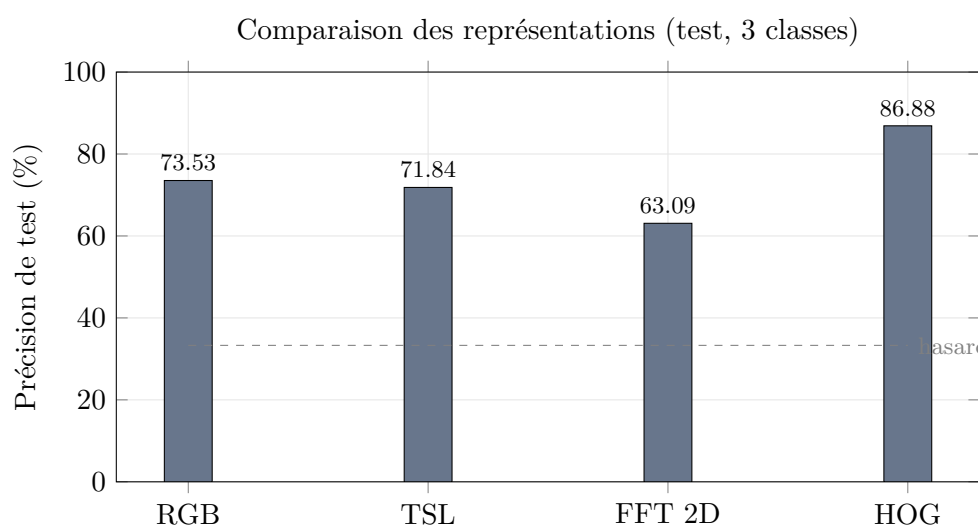


Figure 3: Le HOG améliore nettement la généralisation ; la FFT (amplitude seule) la dégrade.

FFT 2D. Le spectre d'amplitude seul abaisse la précision (63%) : en écartant l'information de phase, il perd une grande part de la structure spatiale discriminante.

HOG. L'histogramme de gradient orienté décrit la *forme* locale (orientation et intensité des contours), une information directement exploitable par un classifieur linéaire. Il porte la précision de test à 86,88%, soit un gain de plus de 13 points sur les pixels bruts. Ce résultat illustre que, à modèle constant, la **qualité de la représentation** est déterminante.

Réserve méthodologique. Le résultat HOG repose sur une exécution unique à 600 itérations ; l'initialisation des poids étant aléatoire, une partie de l'écart peut relever de la variabilité d'initialisation. Une moyenne sur plusieurs exécutions serait nécessaire pour confirmer la valeur

exacte. La tendance — un gain net du HOG sur les autres représentations — reste néanmoins très marquée.

6 Discussion : le plafond du modèle linéaire

L'ensemble des expériences sur pixels bruts (gris, RGB, TSL) converge vers une précision de test de l'ordre de 70–74%, indépendamment des hyperparamètres. Ce plafond n'est pas dû à un réglage insuffisant mais à la **nature linéaire** du modèle : un neurone unique ne trace qu'une frontière de décision linéaire et ne peut séparer des classes qui ne le sont pas dans l'espace des pixels.

Deux leviers permettent de progresser sans changer la nature du modèle : la *multiplication* des neurones (one-vs-all, pour gérer plusieurs classes) et surtout le choix d'une *représentation* dans laquelle les classes deviennent plus séparables (HOG, 86,88%). Franchir durablement ce plafond nécessiterait en revanche une architecture **multi-couches non linéaire** (perceptron multicouche ou réseau convolutif) entraînée par rétropropagation, ce qui dépasse le cadre du neurone unique étudié ici.

7 Conclusion

Nous avons mis en œuvre une chaîne complète de classification d'images fondée sur le neurone fourni, validée d'abord sur les fonctions logiques, puis appliquée à la distinction chat / pas chat, enfin étendue à trois classes. Les expériences confirment le rôle indispensable du mélange et de la normalisation des données, mettent en évidence un sur-apprentissage maîtrisable par arrêt précoce, et caractérisent la robustesse au bruit des différentes fonctions d'activation.

Le résultat marquant est la sensibilité de la performance à la représentation des données : le passage de pixels bruts ($\sim 73\%$) à un descripteur de forme (HOG, $\sim 87\%$) constitue le gain le plus important du projet, là où le réglage fin des hyperparamètres laissait la performance inchangée. Ce constat illustre une idée fondamentale du traitement du signal et de l'apprentissage : *bien représenter la donnée vaut souvent mieux que complexifier le modèle*.

Difficultés rencontrées. (1) La non-convergence de l'apprentissage (boucle infinie pour ReLU/sigmoïde et problèmes non séparables), résolue par l'ajout d'un nombre maximal d'itérations. (2) La cohérence du prétraitement entre train et test : tout changement de format (RGB, TSL, FFT, HOG) devait être appliqué identiquement aux deux ensembles, sous peine d'incohérence dimensionnelle.